
Day 1: Java 21, Spring Boot 3.4 and REST API

Focus

Build a clean Spring Boot REST API using Java 21 and Spring Boot 3.4.

Coverage

- Introduction to microservices
- Monolith vs microservices
- Spring Boot project structure
- Controller, Service and Repository layers
- DTOs
- Basic validation
- Exception handling
- API testing using Postman

Labs

Lab 1: Create Spring Boot Project

Create a new Spring Boot 3.4 project with:

- Spring Web
- Spring Validation
- Lombok
- Spring Boot DevTools

Lab 2: Build Customer Service

Create REST APIs for:

- Add customer
- Get all customers
- Get customer by ID
- Update customer

- Delete customer

Lab 3: Add Validation and Exception Handling

Add:

- Request DTO
 - Response DTO
 - Validation rules
 - Custom exception
 - Global exception handler
-

Day 2: Microservice Communication

Focus

Create two microservices and make them communicate with each other.

Coverage

- Service boundaries
- Customer Service and Order Service design
- REST-based communication
- RestClient or OpenFeign
- Handling service failure
- Spring Boot Actuator basics

Labs

Lab 1: Create Order Service

Create a new Spring Boot microservice for managing orders.

Lab 2: Call Customer Service from Order Service

When creating an order:

- Accept customer ID
- Call Customer Service
- Check if customer exists
- Create order only if customer is valid

Lab 3: Add Actuator

Add health check endpoints:

/actuator/health

/actuator/info

Lab 4: Handle Failure

Stop Customer Service and test Order Service.

Add proper error response when Customer Service is unavailable.

Day 3: Kafka for Event-Driven Microservices

Focus

Use Kafka to send order events between microservices.

Coverage

- What is Kafka?
- Producer and consumer
- Topic
- Consumer group
- Event-driven communication
- Publishing events from Spring Boot
- Consuming events in Spring Boot

Labs

Lab 1: Run Kafka Using Docker

Run Kafka locally using Docker Compose.

Create topic:

order-created-topic

Lab 2: Publish Order Event

When an order is created, publish an event to Kafka.

Sample event:

```
{
  "orderId": 101,
  "customerId": 1,
  "amount": 2500,
  "status": "CREATED"
}
```

Lab 3: Create Notification Service

Create a Notification Service that consumes order-created-topic and prints a notification message.

Lab 4: Add Billing Consumer

Create a Billing Service that also consumes the same event using a separate consumer group.

Day 4: Apache Camel for Enterprise Integration

Focus

Use Apache Camel to process files and connect systems.

Coverage

- What is Apache Camel?

- Camel route
- Camel endpoint
- Camel processor
- File processing
- Basic data transformation
- REST to Kafka integration

Labs

Lab 1: Create Camel Spring Boot Project

Create a Apache Camel project using Spring Boot.

Create a basic route:

timer -> log

Lab 2: File Processing Route

Create a route:

input folder -> process file -> output folder

The route should:

- Read files from input folder
- Log file content
- Move processed files to output folder

Lab 3: CSV to JSON Transformation

Read customer CSV data and convert it into JSON.

Sample CSV:

```
id,name,email
1,Rahul,rahul@example.com
2,Neha,neha@example.com
```

Lab 4: REST to Kafka Route

Create a Camel route:

REST endpoint -> Kafka topic

Endpoint:

POST /camel/orders

The route should accept an order request and publish it to Kafka.

Day 5: Docker, Kubernetes and Capstone

Focus

Containerize and deploy the application.

Coverage

- Docker basics
- Dockerfile for Spring Boot
- Docker Compose
- Kubernetes basics
- Pod, Deployment and Service
- Deploying microservices
- Viewing logs and troubleshooting

Labs

Lab 1: Dockerize Services

Create Dockerfiles for:

- Customer Service
- Order Service
- Notification Service
- Camel Integration Service

Lab 2: Run Using Docker Compose

Run the complete system using Docker Compose:

- Customer Service
- Order Service
- Notification Service
- Kafka
- Camel Integration Service

Lab 3: Deploy to Kubernetes

Create basic Kubernetes YAML files:

- Deployment
- Service
- ConfigMap

Deploy services using:

```
kubectl apply -f .  
kubectl get pods  
kubectl get services  
kubectl logs <pod-name>
```

Lab 4: Scale Order Service

Scale Order Service:

```
kubectl scale deployment order-service --replicas=2
```

Capstone Project

Project Name

Enterprise Order Processing Platform

Objective

Build a microservices-based order processing system using Spring Boot, Kafka, Apache Camel, Docker and Kubernetes.

Services

Customer Service

- Add customer
- View customer
- Update customer
- Delete customer

Order Service

- Create order
- Validate customer
- Publish order-created event to Kafka

Notification Service

- Consume order-created event
- Print notification message

Billing Service

- Consume order-created event
- Print billing message

Camel Integration Service

- Accept external order request
- Process file input
- Publish order message to Kafka

Final Outcome

By the end of the training, participants will be able to:

- Build REST APIs using Spring Boot
- Create basic microservices
- Connect microservices using REST

- Use Kafka for event communication
 - Build Apache Camel routes
 - Dockerize Spring Boot services
 - Deploy services to Kubernetes
-

Tools Required

- Java 21
- Spring Boot 3.4
- Maven
- IntelliJ IDEA or Eclipse
- Postman
- Docker Desktop
- Kafka using Docker Compose
- Apache Camel
- Kubernetes using Docker Desktop, Minikube or Kind
- kubectl
- GitHub